

LAB 01 — From Text Processing to Search

Môn học: Xử lý ngôn ngữ tự nhiên và ứng dụng | Học kỳ: Học kỳ I - 2026

Giảng viên / TA: Phạm Ngọc Hải ▪ Hạn nộp: 23:59 - 24/09/2026

1. Tổng quan

1.1. Chủ đề

Text Processing, TF-IDF Representation và Document Search

1.2. Bối cảnh

Trong phần lý thuyết, sinh viên đã được giới thiệu:

- Sparse vector representations of documents.
- Count vector representation.
- Normalization of count vectors.
- TF-IDF.
- Document processing pipeline:

```
Tokenizer
  ↓
CountVectorizer
  ↓
IDF
  ↓
TF-IDF representation
```

- Demo trên corpus gồm khoảng **30.000 documents**.

Do đó, buổi lab **không nhằm mục tiêu dạy lại TF-IDF** hoặc yêu cầu sinh viên đơn giản gọi `TfidfVectorizer()`.

Mục tiêu của lab là chuyển kiến thức lý thuyết thành khả năng:

tính toán → kiểm chứng → xây dựng → ứng dụng → đánh giá → phân tích giới hạn của một NLP representation.

1.3. Câu hỏi trung tâm

Một quyết định trong text processing ảnh hưởng như thế nào đến representation của document và chất lượng của một hệ thống tìm kiếm?

Buổi học sẽ đi theo chuỗi:

```
Theory recap
  ↓
Calculation
  ↓
Prediction
  ↓
Experiment
  ↓
Implementation
  ↓
Application: Search
  ↓
Evaluation
  ↓
Error analysis
  ↓
```

Reflection



Limitation → Next topic

2. Learning Outcomes

Sau khi hoàn thành lab, sinh viên có thể:

LO1 — Explain

Giải thích được cách một document được chuyển từ text sang sparse TF-IDF vector.

LO2 — Calculate

Tự tính được:

- term frequency;
- document frequency;
- inverse document frequency;
- TF-IDF;
- cosine similarity.

LO3 — Implement

Tự triển khai một phiên bản đơn giản của:

- term counting;
- TF;
- IDF;
- TF-IDF;
- cosine similarity.

LO4 — Experiment

Thiết kế và thực hiện các thí nghiệm để kiểm tra ảnh hưởng của preprocessing đối với representation.

LO5 — Apply

Sử dụng TF-IDF để xây dựng một hệ thống document search đơn giản trên corpus 30K documents.

LO6 — Evaluate

Đánh giá hệ thống bằng các metric retrieval phù hợp và thực hiện error analysis.

LO7 — Reflect

Nhận diện được giới hạn của lexical representation và giải thích tại sao các phương pháp representation tiếp theo là cần thiết.

3. Cấu trúc buổi lab

Phần	Nội dung	Thời lượng dự kiến
A	Theory recap	15 phút
B	Calculation exercises	25 phút
C	Prediction trước experiment	10 phút
D	Experiment 1: Sparse representation	25 phút
E	Core implementation	35 phút

Phần	Nội dung	Thời lượng dự kiến
F	Experiment 2: Preprocessing ablation	30 phút
G	Application: Document search	35 phút
H	Evaluation và error analysis	30 phút
I	Reflection và learning check	15 phút

4. Part A — Theory Recap

4.1. Count Vector Representation

Cho corpus:

```
D1 = "machine learning for healthcare"
D2 = "deep learning for healthcare"
D3 = "machine learning for images"
```

Vocabulary:

```
[machine, learning, healthcare, deep, for, images]
```

Mỗi document được biểu diễn bằng vector số:

```
D1 → [1, 1, 1, 0, 1, 0]
D2 → [0, 1, 1, 1, 1, 0]
D3 → [1, 1, 0, 0, 1, 1]
```

Đây là **sparse vector representation**.

4.2. Term Frequency

Với document (d) và term (t):

$$tf(t, d) = \frac{c(t, d)}{\sum_{t'} c(t', d)}$$

trong đó:

- $c(t,d)$: số lần term (t) xuất hiện trong document (d);
- mẫu số là tổng số term trong document.

4.3. Document Frequency

$$df(t) = |\{d : t \in d\}|$$

Tức là số documents chứa term (t).

4.4. Inverse Document Frequency

Một dạng thường dùng:

$$idf(t) = \log \frac{N}{df(t)}$$

với (N) là tổng số documents.

Một số thư viện sử dụng biến thể smoothing:

$$idf(t) = \log \frac{1 + N}{1 + df(t)} + 1$$

Sinh viên cần chú ý rằng **các convention khác nhau có thể tạo ra numerical result khác nhau**.

4.5. TF-IDF

$$tfidf(t, d) = tf(t, d) \times idf(t)$$

Ý tưởng:

Một term quan trọng đối với document nếu nó xuất hiện nhiều trong document đó nhưng không xuất hiện quá phổ biến trong toàn corpus.

4.6. Cosine Similarity

Với hai vector (x) và (y):

$$\cos(x, y) = \frac{x^\top y}{\|x\|_2 \|y\|_2}$$

Cosine similarity đo mức độ giống nhau về **hướng** của hai vector.

5. Part B — Calculation Exercises

Phần này phải được hoàn thành **trước khi sử dụng code để kiểm tra kết quả**.

Exercise 1 — Count Vector

Cho corpus:

```
D1 = "cat eats fish"
D2 = "dog eats fish"
D3 = "cat likes fish"
```

Vocabulary được sắp xếp theo thứ tự:

```
[cat, dog, eats, fish, likes]
```

Hãy tính count vector của:

- (D_1);
- (D_2);
- (D_3).

Exercise 2 — TF

Với (D_1):

```
cat eats fish
```

hãy tính:

$$tf(cat, D_1)$$

$$tf(eats, D_1)$$

$$tf(fish, D_1)$$

Kiểm tra:

$$\sum_t tf(t, D_1) = 1$$

Exercise 3 — IDF

Corpus có:

$$N = 3$$

và:

```
df(cat) = 2  
df(dog) = 1  
df(eats) = 2  
df(fish) = 3  
df(likes) = 1
```

Sử dụng:

$$idf(t) = \log \frac{N}{df(t)}$$

hãy tính IDF của từng term.

Sau đó trả lời:

Term nào có IDF thấp nhất? Vì sao?

Exercise 4 — TF-IDF

Tính TF-IDF của (D₁):

```
D1 = "cat eats fish"
```

cho cả ba term:

```
cat  
eats  
fish
```

Sau đó trả lời:

Tại sao fish xuất hiện trong mọi document nhưng TF-IDF của nó bằng 0 theo công thức trên?

Exercise 5 — Cosine Similarity

Cho:

$$x = [1, 1, 1]$$

và:

$$y = [1, 1, 0]$$

Tính:

$$\cos(x, y)$$

Sau đó giải thích bằng trực giác:

Hai documents có hai term giống nhau trên ba term tổng cộng. Tại sao cosine similarity không bằng (2/3)?

Exercise 6 — Prediction

Cho:

```
D1 = "medical image classification"
D2 = "medical image analysis"
D3 = "natural language processing"
```

Query:

```
Q = "medical image classification"
```

Không dùng code, hãy dự đoán:

1. Document nào có similarity cao nhất?
2. Document nào có similarity thấp nhất?
3. Term nào có thể có giá trị IDF thấp?
4. Nếu bỏ IDF và chỉ sử dụng count vector thì ranking có thay đổi không?

Ghi lại câu trả lời trước khi chạy experiment.

! Quy tắc của phần tính toán

Không sử dụng AI hoặc code để tính trước.

Mục tiêu không phải kiểm tra khả năng tính toán số học, mà là kiểm tra xem sinh viên có thể **dự đoán hành vi của representation trước khi quan sát kết quả** hay không. Sau khi hoàn thành, có thể sử dụng Python để kiểm chứng.

6. Part C — Prediction Before Experiment

Trước khi mở corpus 30K documents, sinh viên phải đưa ra ít nhất ba prediction.

Prediction 1 — Vocabulary

Nếu corpus có 30K documents, vocabulary sẽ có khoảng bao nhiêu unique terms?

Prediction 2 — Sparsity

TF-IDF matrix sẽ dense hay sparse?

Tỷ lệ zero entries có thể lớn đến mức nào?

Prediction 3 — Search

Với một query bất kỳ:

Các documents đứng đầu kết quả tìm kiếm có nhất thiết là documents gần nghĩa nhất không?

Sinh viên ghi prediction vào:

```
prediction.md
```

Kết quả thực nghiệm chỉ được xem **sau khi prediction đã được ghi lại**.

7. Part D — Experiment 1: Inspect the Sparse Representation

7.1. Dataset

Sử dụng corpus gồm khoảng **30.000 documents** đã được giới thiệu trong lecture.

Không thay đổi dataset ở experiment này.

7.2. Xây dựng pipeline

Sử dụng pipeline đã học:

```
Raw documents
  ↓
Tokenizer
  ↓
CountVectorizer
  ↓
TF
  ↓
IDF
  ↓
TF-IDF matrix
```

7.3. Kiểm tra kích thước

Ghi lại:

```
Number of documents = ?
Vocabulary size      = ?
Matrix shape         = ?
```

Nếu:

$$X \in \mathbb{R}^{N \times V}$$

hãy xác định:

- (N);
- (V).

7.4. Kiểm tra sparsity

Tính:

$$S = 1 - \frac{\text{nnz}(X)}{N \times V}$$

trong đó ($\text{nnz}(X)$) là số phần tử khác 0.

Trả lời:

Tại sao một document chỉ sử dụng một phần rất nhỏ vocabulary nhưng vector vẫn có chiều (V)?

7.5. Inspect vocabulary

Tìm:

- 20 terms phổ biến nhất theo document frequency;
- 20 terms có IDF cao nhất;
- 20 terms có TF-IDF cao nhất trong một document được chọn.

So sánh ba danh sách.

Câu hỏi

Một term xuất hiện rất nhiều trong corpus có nhất thiết có TF-IDF cao không?

Một term có IDF cao có nhất thiết có TF-IDF cao trong mọi document không?

8. Part E — Core Implementation

8.1. Mục tiêu

Tự xây một phiên bản TF-IDF tối giản trên một corpus nhỏ.

Không sử dụng trực tiếp:

```
TfidfVectorizer()
```

cho phần implementation cốt lõi.

8.2. Các hàm cần xây dựng

```
build_vocabulary()  
compute_counts()  
compute_tf()  
compute_idf()  
compute_tfidf()  
cosine_similarity()
```

8.3. Corpus kiểm thử

Sử dụng:

```
D1 = "cat eats fish"  
D2 = "dog eats fish"  
D3 = "cat likes fish"
```

8.4. Unit tests

Mỗi hàm phải có ít nhất một test.

Ví dụ:

```
assert abs(tf_cat - expected_value) < 1e-9
```

8.5. So sánh với thư viện

Sau khi implementation hoàn thành, mới sử dụng thư viện để kiểm tra.

So sánh:

```
Student implementation
      vs
Reference implementation
```

Nếu kết quả khác nhau, **không sửa code ngay theo gợi ý của AI**.

Trước tiên phải xác định:

Khác nhau ở công thức, convention, vocabulary ordering, normalization hay implementation?

9. Part F — Experiment 2: Preprocessing Ablation

Đây là experiment quan trọng nhất của buổi lab.

Mục tiêu:

Preprocessing không chỉ là cleaning; nó là một modeling decision.

9.1. Pipeline A — Minimal

```
Raw text
  ↓
Lowercase
  ↓
Tokenization
```

9.2. Pipeline B — Normalized

```
Raw text
  ↓
Lowercase
  ↓
Punctuation normalization
  ↓
Tokenization
  ↓
Stopword handling
```

9.3. Pipeline C — Extended

```
Raw text
  ↓
Normalization
  ↓
Subword tokenization
```

9.4. So sánh

Với mỗi pipeline, đo:

Metric	Pipeline A	Pipeline B	Pipeline C
Vocabulary size			
Average tokens/document			

Metric	Pipeline A	Pipeline B	Pipeline C
Matrix sparsity			
OOV rate			
Search performance			

9.5. Câu hỏi phân tích

Trả lời bằng kết quả thực nghiệm, không chỉ bằng lý thuyết:

1. Lowercasing làm thay đổi vocabulary như thế nào?
2. Stopword removal có luôn cải thiện representation không?
3. Việc loại punctuation có thể làm mất thông tin gì?
4. Pipeline nào tạo ra sparse matrix nhất?
5. Pipeline nào cho search tốt nhất?
6. Search tốt hơn có đồng nghĩa với vocabulary nhỏ hơn không?

⚠ Không được giả định preprocessing càng nhiều càng tốt

Một preprocessing operation chỉ nên được giữ lại nếu có lý do và evidence cho thấy nó phù hợp với nhiệm vụ.
Mục tiêu của experiment là **kiểm chứng**, không phải chứng minh rằng một pipeline cụ thể luôn tốt hơn.

10. Part G — Application: Build a Document Search Engine

10.1. Bài toán

Xây dựng một hệ thống tìm kiếm đơn giản trên corpus 30K documents.

Input:

User query

Output:

Top-K relevant documents

10.2. Pipeline

```
30K Documents
  ↓
TF-IDF Index
  ↓
User Query
  ↓
TF-IDF Query Vector
  ↓
Cosine Similarity
  ↓
Ranking
  ↓
Top-K Documents
```

10.3. Query examples

Có thể sử dụng các query như:

```
"medical image classification"
"transformer language model"
"deep learning healthcare"
"natural language processing"
```

10.4. Kết quả cần hiển thị

Với mỗi query:

Rank	Document ID	Similarity	Document preview
1			
2			
3			
4			
5			

11. Part H — Evaluation

11.1. Tạo evaluation set

Chuẩn bị một tập nhỏ gồm khoảng **5-10 queries**.

Mỗi query cần có relevance labels.

Ví dụ:

```
Query 1:
"medical image classification"

Relevant:
D12
D45
D78
```

Không nhất thiết phải gán nhãn toàn bộ 30K documents.

11.2. Precision@K

$$P@K = \frac{\text{\#relevant documents retrieved}}{K}$$

Tính:

$$P@5$$

11.3. Recall@K

$$R@K = \frac{\text{\#relevant documents retrieved}}{\text{\#relevant documents}}$$

Tính:

$$R@5$$

11.4. Mean Reciprocal Rank

Nếu document relevant đầu tiên của query (q) xuất hiện ở rank (r_q):

$$MRR = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{r_q}$$

Không cần sử dụng quá nhiều metric trong lab đầu tiên.

Ba metric chính:

- Precision@5;
- Recall@5;
- MRR.

12. Part I — Error Analysis

Chọn:

- 2 queries có kết quả tốt;
- 2 queries có kết quả kém.

Với mỗi query, phân tích:

Query

...

Expected relevant documents

...

Retrieved documents

1. ...
2. ...
3. ...
4. ...
5. ...

Analysis

Trả lời:

1. Vì sao document đứng đầu?
2. Những từ nào đóng góp nhiều vào similarity?
3. Có lexical overlap không?
4. Có relevant document nào bị bỏ sót không?
5. Failure này xuất phát từ:
 - preprocessing;
 - TF;
 - IDF;
 - vocabulary;
 - lexical matching;
 - hay nguyên nhân khác?

Failure case quan trọng nhất

Chọn một failure case và giải thích thật kỹ.

Ví dụ:

```
Query:
"heart attack treatment"

Relevant document:
"myocardial infarction therapy"
```

Nếu hệ thống không xếp document này cao, hãy giải thích:

Hai biểu thức có thể gần nhau về nghĩa nhưng không có lexical overlap tương ứng.

13. Part J — From Failure to the Next NLP Representation

Đây là phần kết nối với các buổi tiếp theo.

TF-IDF biểu diễn document dựa chủ yếu trên **lexical statistics**.

Điều này tạo ra một hạn chế:

```
"heart attack"
≠
"myocardial infarction"
```

nếu hai biểu thức không chia sẻ đủ token.

Trong khi đó, con người có thể nhận ra:

```
heart attack
≈
myocardial infarction
```

về mặt ngữ nghĩa.

Câu hỏi cuối buổi

Làm thế nào để biểu diễn được similarity về nghĩa thay vì chỉ similarity về từ?

Sinh viên không cần giải quyết vấn đề ngay.

Nhưng phải đưa ra ít nhất **một hypothesis**.

Ví dụ:

Có thể cần một representation trong đó các từ xuất hiện trong những context tương tự được biểu diễn bằng các vector gần nhau.

Đây chính là động lực để chuyển sang:

```
TF-IDF
↓
Distributional representation
↓
Word Embedding
↓
Contextual Embedding
↓
Transformer
```

14. AI Usage Policy

AI được phép sử dụng trong lab, nhưng phải phân biệt **hỗ trợ học tập** và **thay thế quá trình suy nghĩ**.

Được phép

Sinh viên có thể sử dụng AI để:

- giải thích documentation;
- tìm nguyên nhân của lỗi Python;
- giải thích warning/error;
- đề xuất cách visualize;
- gợi ý cách tối ưu code;
- so sánh hai API.

Phải khai báo

Nếu AI sinh hoặc sửa code:

```
AI contribution:  
- Generated initial implementation of cosine similarity.  
- Suggested sparse matrix optimization.  
- I modified the implementation and verified it against ...
```

Không sử dụng AI trong

Các phần cần chứng minh năng lực cá nhân:

- prediction trước experiment;
- calculation exercises trước khi kiểm chứng;
- giải thích numerical results;
- error analysis;
- reflection;
- oral learning check.

i Nguyên tắc

Mục tiêu không phải **“không được dùng AI”**.

Mục tiêu là:

Có thể dùng AI để làm việc nhanh hơn, nhưng không thể dùng AI để thay thế việc hiểu kết quả.

15. Learning Check

Cuối buổi, mỗi sinh viên phải có khả năng trả lời ngắn gọn các câu hỏi sau.

Question 1

Tại sao TF-IDF tạo ra sparse representation?

Question 2

Tại sao một term xuất hiện trong hầu hết documents có IDF thấp?

Question 3

Tại sao một term có IDF cao chưa chắc có TF-IDF cao trong một document?

Question 4

Tại sao cosine similarity phù hợp với document vectors?

Question 5

Tại sao preprocessing có thể thay đổi search result?

Question 6

Một failure case của TF-IDF search mà em quan sát được là gì?

Question 7

Failure case đó gợi ý nhu cầu về representation nào tiếp theo?

16. Reflection

Nộp một file reflection.md với tối đa khoảng 500 từ.

Trả lời:

1. Prediction nào của em sai?
 2. Kết quả nào bất ngờ nhất?
 3. Experiment nào cung cấp evidence mạnh nhất?
 4. Failure case quan trọng nhất là gì?
 5. Nếu được xây lại search engine, em sẽ thay đổi điều gì?
 6. AI đã được sử dụng ở những phần nào và đóng góp cụ thể là gì?
-

17. Deliverables

Cấu trúc thư mục:

```
lab01/  
├── README.md  
├── calculations.md  
├── prediction.md  
├── implementation.py  
├── experiments.ipynb  
├── results.csv  
└── reflection.md
```

calculations.md

Các bài tính tay.

prediction.md

Prediction trước experiment.

implementation.py

Core TF-IDF và cosine similarity.

experiments.ipynb

Các experiment trên corpus 30K.

results.csv

Kết quả retrieval/evaluation.

reflection.md

Phân tích và reflection.

18. Assessment Rubric

Thành phần	Điểm
Theory recap & calculation	10
Prediction	10
Core implementation	20
Preprocessing experiment	15
Search application	15
Quantitative evaluation	15
Error analysis	10
Reflection / learning check	5
Tổng	100

Nguyên tắc chấm

Không đánh giá chủ yếu dựa trên:

- số dòng code;
- notebook đẹp;
- sử dụng thư viện phức tạp;
- accuracy cao một cách đơn thuần.

Đánh giá chủ yếu dựa trên:

understanding + experimental reasoning + evidence + interpretation.

19. Instructor Notes

Không cho sẵn toàn bộ implementation

Giảng viên cung cấp:

- corpus;
- starter notebook;
- function signatures;
- expected input/output;
- một vài test cases.

Nhưng không cung cấp implementation của:

```
compute_tf()
compute_idf()
compute_tfidf()
cosine_similarity()
```

Không yêu cầu implementation quá lớn

Mục đích là:

hiểu algorithm

không phải:

xây lại scikit-learn.

Không dùng search evaluation quá phức tạp

Lab 1 cần:

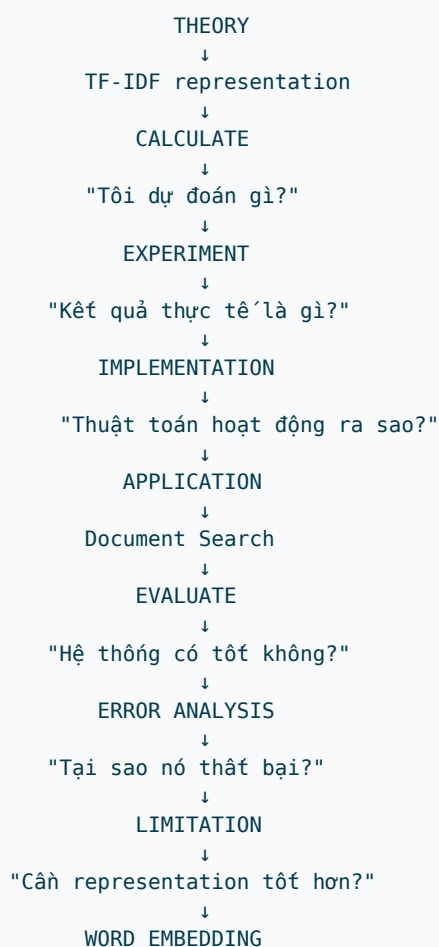
P@5
Recall@5
MRR

Sau này khi học dense retrieval có thể mở rộng sang:

- Recall@K;
- MRR;
- nDCG;
- reranking evaluation.

20. Kết cấu tư duy của LAB 01

Toàn bộ buổi lab có thể tóm tắt thành:



Ý tưởng cốt lõi

Mục tiêu là để sinh viên trải qua một chu trình nghiên cứu nhỏ:

Hiểu một mô hình → tính được nó → tự implement một phần → đưa vào bài toán thực tế → đo lường → tìm failure → từ failure suy ra nhu cầu về phương pháp tiếp theo.